

Assignment-1 Template

STUDENT NAME

Mohammed Alosaily

CLASS / SECTION

201

1 Functional Requirement

What the system does: "When <trigger>, the system <behavior> and <result>."

When a librarian types a student ID and a book ID and picks "Borrow", the system checks that the student exists, the book is on the shelf and the student has fewer than 3 books, and then it saves the loan with a due date 14 days later and shows "OK: ... Due back on <date>". (Idea: School library borrowing app)

2 Non-Functional Requirement

How well it does it — give a measurable target, not the word "fast".

Borrowing or returning a book shows the result in under 1 second for a library of up to 5,000 books. A new librarian should be able to borrow a book in 3 steps or less from the menu, and wrong input (like a fake ID) never crashes the program, it just prints an error message.

3 Responsibility

The ONE job each module is trusted to do. One sentence, no "and also".

Catalog: keeps the list of books and whether each one is on the shelf. Members: keeps the list of students allowed to borrow. LoanManager: applies the borrowing rules (3-book limit, 14-day due date, overdue). LibraryMenu: shows the menu to the librarian and prints the results.

4 Boundary

What the module exposes to others, and what stays hidden inside it.

Each class exposes only methods, like `add_book()`, `is_available()`, `is_member()`, `borrow()`, `return_book()` and `overdue_list()`. The data (the `_books`, `_students` and `_loans` dictionaries) is hidden inside each class with an underscore, and the limits (`MAX_BOOKS`, `LOAN_DAYS`) are only used inside `LoanManager`.

5 Cohesion

Why these parts belong in one module: they all change for the same reason.

Everything in `LoanManager` is about the borrowing rules: `borrow`, `return` and `overdue` all change only if the school changes its loan policy (e.g. 21 days instead of 14). `Catalog` only changes if we store more info about books, and `Members` only if we store more info about students, so each module has one reason to change.

6 Coupling

Which module depends on which, and how you keep that link weak.

`LibraryMenu` -> `LoanManager`, `Catalog`, `Members`. `LoanManager` -> `Catalog`, `Members`. `Catalog` and `Members` depend on nothing. To keep it weak, modules only call each other's public methods (never the dictionaries), and objects are passed in through the constructor, so I can swap `Catalog` for a database later without touching `LoanManager`.